



Lesson 2: Data Exploration and Preparation

Technologies for Artificial Intelligence

Fabio Antonini, Università degli Studi dell'Aquila

Before we start

Exercise 1 was set last week. We discuss it now.

- The wine-quality workflow, end to end
- What counts is **methodology**, not accuracy
- A show of hands: did you predict a class, or a number?

What today builds on

- Lesson 1's rule: **nothing is learned before the split**
- Lesson 1's tool: `Pipeline`, which enforces that rule in the code instead of relying on memory
- Today: both, on messy data - gaps, extreme values, text categories

Today

- Exploratory analysis
- Missing values
- Outliers
- Scaling and encoding
- Feature engineering and pipelines
- Leakage that raises no error and looks like a good result

One dataset, all lesson long

2000 synthetic telecom customers. Will they **churn** (cancel their contract)?

- Numeric: tenure, monthly charges, age, support calls
- Categorical: contract type, region, zip code (493 different values)
- Missing values, extreme values, and one column built to have **no relation** to churn

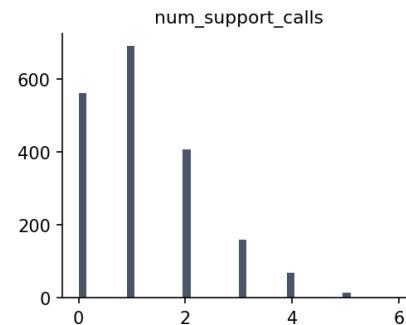
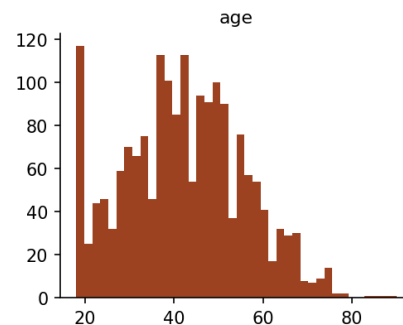
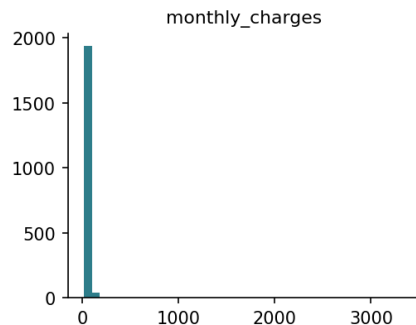
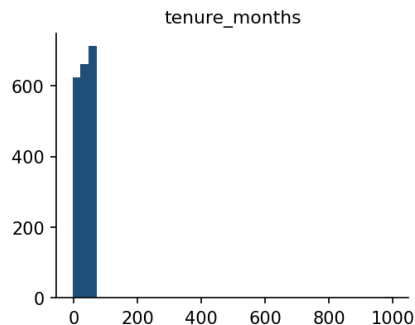
Look before you touch anything

First, always: `.info()`, `.describe()`, and the share of each class.

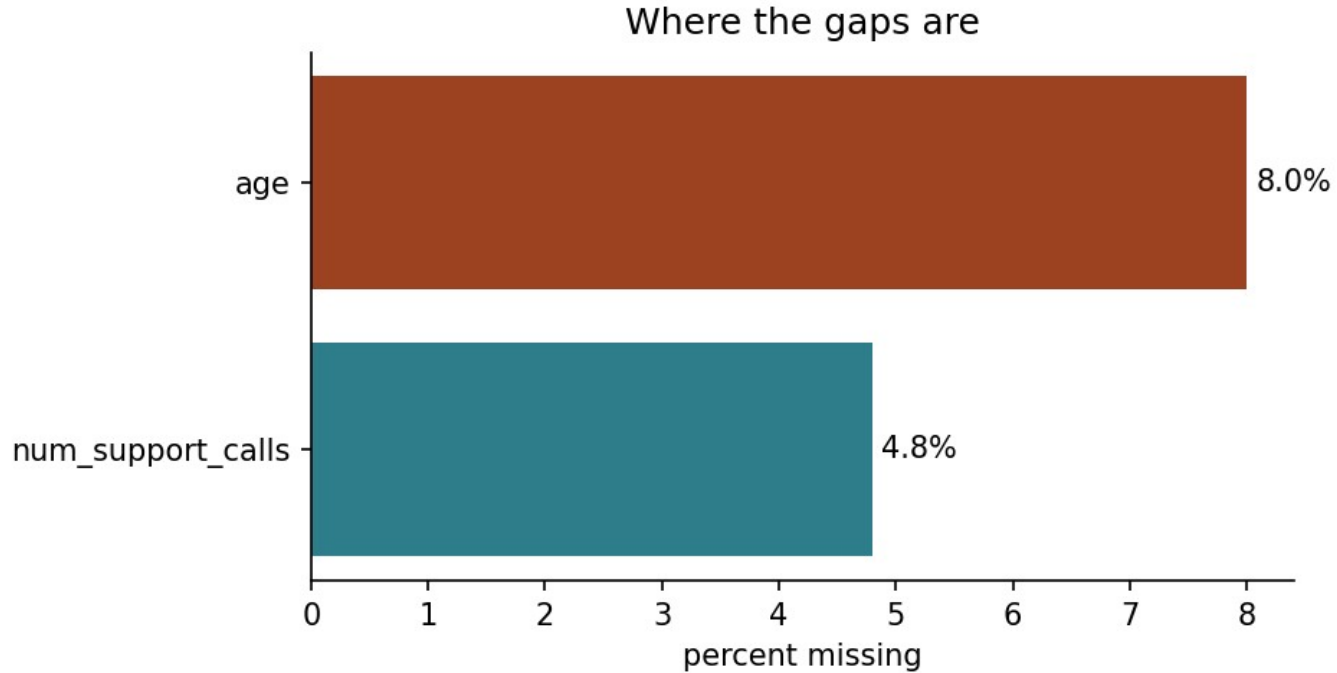
- 2000 rows, 8 columns, numbers and text
- Churn rate **19.4%**: always predicting “no churn” is right 80.6% of the time
- `tenure_months` reaches 999 months, 83 years: an error in the data

Outliers squash two of these four

Raw distributions, before any cleaning



Where the gaps are



Three reasons a value is missing

Three reasons a value can be missing

MCAR

missing completely at random

P(missing)



depends on nothing

MAR

missing at random

P(missing)



depends on an
OBSERVED column

MNAR

missing not at random

P(missing)



depends on the
MISSING value itself

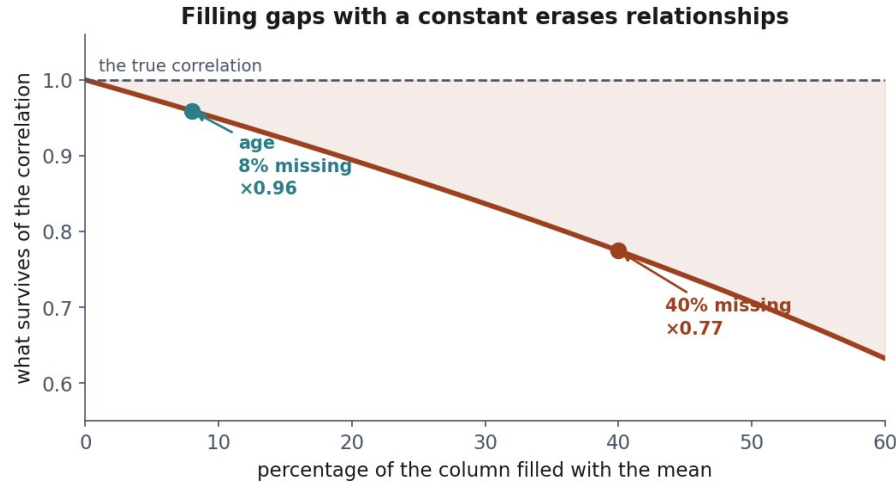
age: missing completely at random

MCAR, for short: some customers left it blank, for no reason related to any column.

- Filling the gaps with the mean leaves the mean of age unbiased
- So is filling free? The next slide shows what it costs

What mean imputation actually costs

Filling with the mean weakens every correlation age has with other columns, even under MCAR.



Support calls: missing at random

MAR, for short: long-tenure customers leave it blank more often.

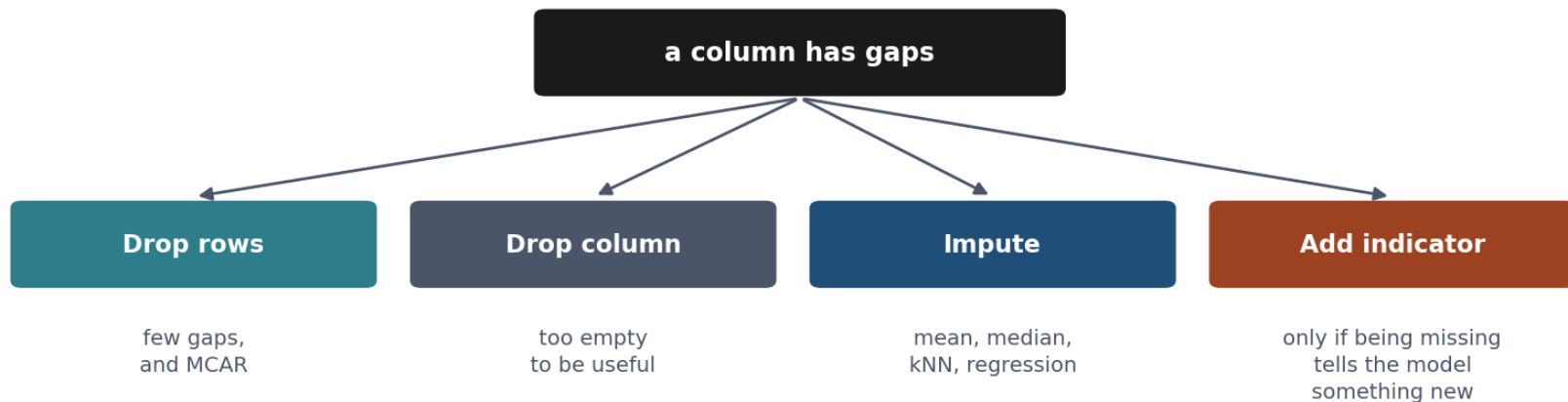
- Dropping those rows would remove mostly long-tenure customers
- A **missingness indicator** (1 if missing, 0 if not) adds nothing here: `tenure_months` already tells the model which rows tend to have a gap

What to do about a gap

- **Drop rows:** only if few, and only if MCAR
- **Drop the column:** if it is missing too often to help
- **Impute:** mean/median or most-frequent with `SimpleImputer`, neighbours with `KNNImputer`
- **Add a missingness indicator:** only if *being missing* tells the model something the other columns cannot - typically when missing not at random (MNAR)

Choosing among the four

Which one you pick is a question about why the data is missing



the mechanism decides, and every option except dropping the column learns a statistic, so it is fitted on the training data only

Extreme does not mean wrong

- `tenure_months` runs **-3 to 999**; ordinary customers sit at 0 to 72
- Three reasons: a **recording error**, a **rare but genuine** value, a **different population**
- A monthly charge of 3,344.7 may be a typo or a corporate account: the numbers alone cannot tell
- Two rules follow, one on standard deviations, one on quartiles: both only **flag** a value for a person to check

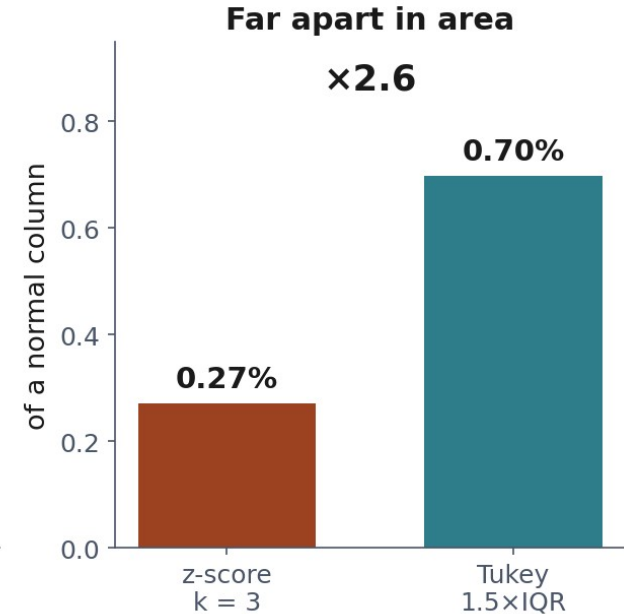
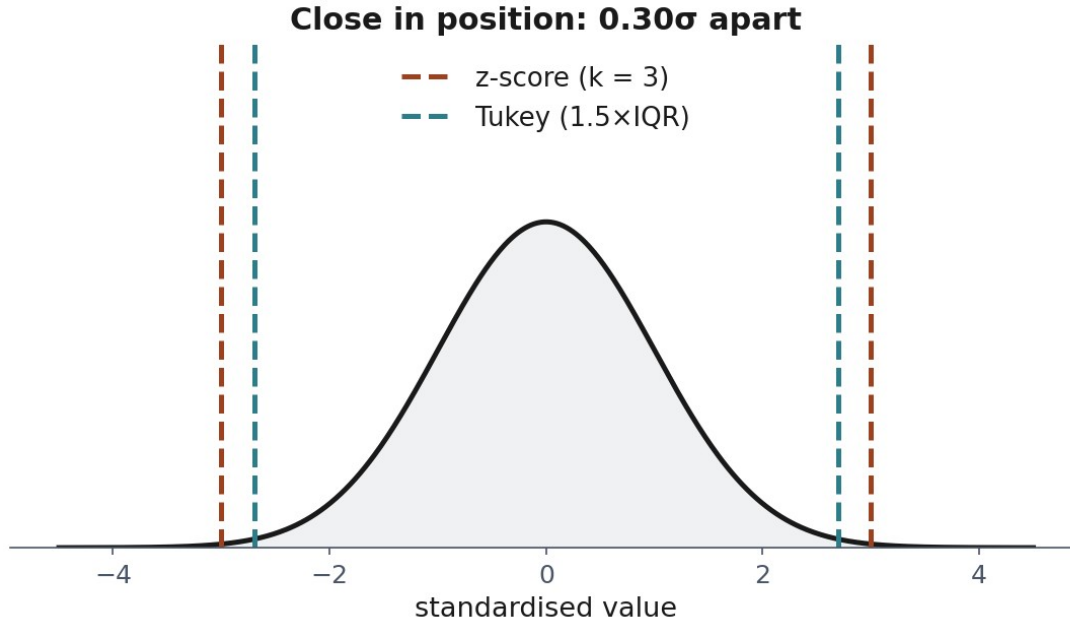
Rule 1: distance in standard deviations, flag beyond $k = 3$

$$Z_i = \frac{x_i - \bar{x}}{s}$$

Rule 2: interquartile range (IQR)

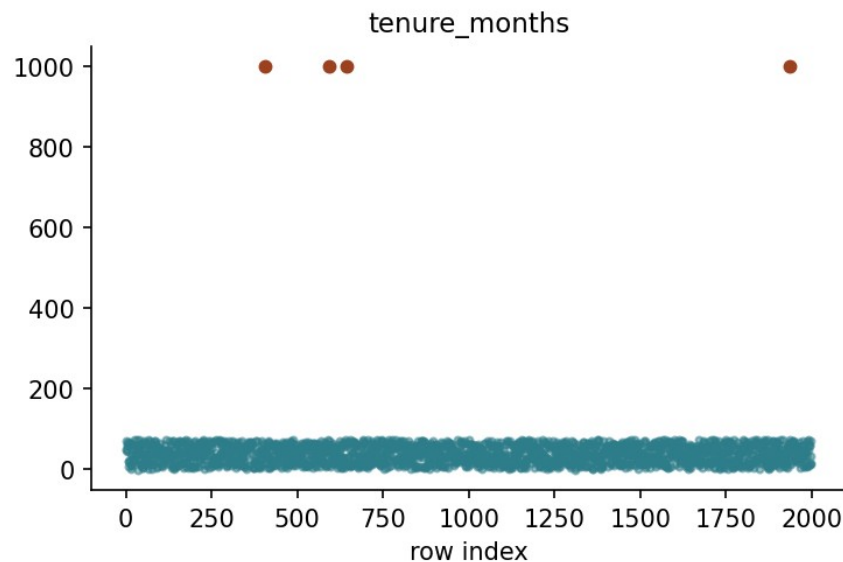
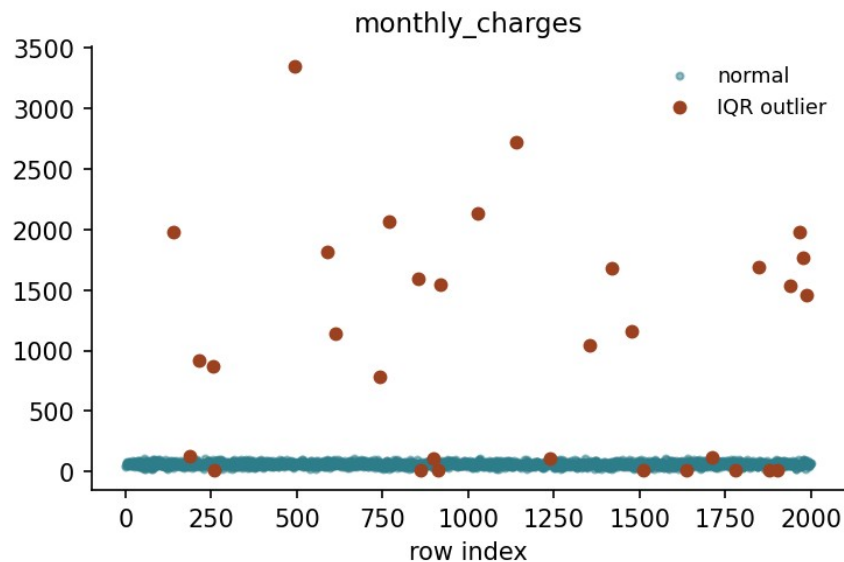
$$[Q_1 - 1.5 \text{ IQR}, \quad Q_3 + 1.5 \text{ IQR}]$$

Close fences, very different counts



Quartile rule: 12 false alarms

What the IQR rule catches



What neither rule can tell you

`tenure_months` of **-3** is impossible. Neither rule flags it.

- -3 is not *extreme* relative to a column spanning 0-72
- It is not unusual, it is impossible: no tenure is negative. Checking validity needs a rule about the domain, not about the distribution

Once something is flagged

The rules only find candidates. What to do with them is decided by someone who knows the **domain**.

- **Cap** it at the rule's upper limit: 3,344.7 becomes 110.0 (*winsorising*)
- **Remove** the row
- **Correct** it, if the true value is recoverable
- **Leave** it: unusual is not the same as wrong

Notebook 1, live

section, and its slides	you call (pandas, no model yet)
first look, 6	<code>df.info()</code> , <code>df.describe()</code>
distributions, 7	<code>df.hist()</code>
missingness, 8-12	<code>df.isna().mean()</code> , and the mean before and after filling
outliers from scratch, 15-19	<code>.mean()</code> and <code>.std()</code> for rule 1, <code>.quantile([0.25, 0.75])</code> for rule 2
correlations, categories	<code>df.corr()</code> , <code>.value_counts()</code>

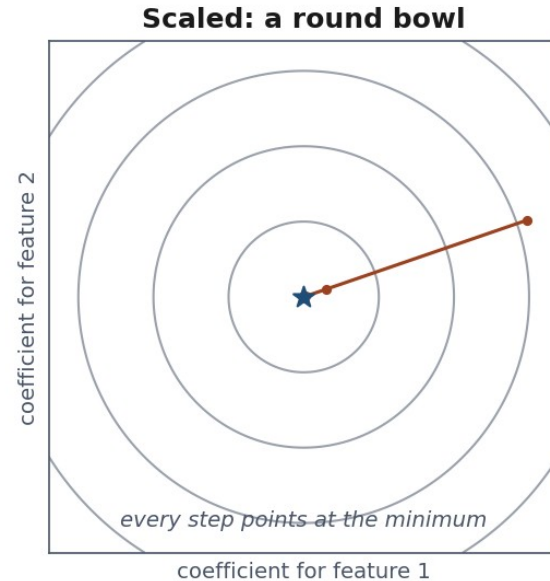
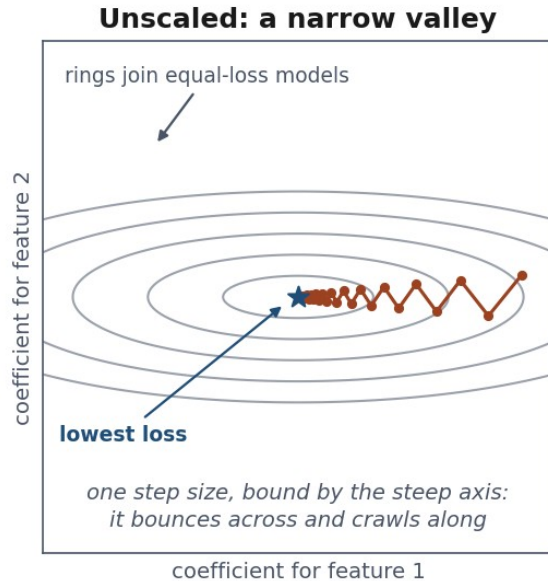
Break

Why scale?

- `tenure_months: 0-72. monthly_charges: 15-128`
- **Training** = choosing the model's numbers to make Lesson 1's average loss small
- **Gradient descent** does it by repeated small steps downhill on that loss
- The same step size - the **learning rate** - is used for every column at every step. Can one size suit both?

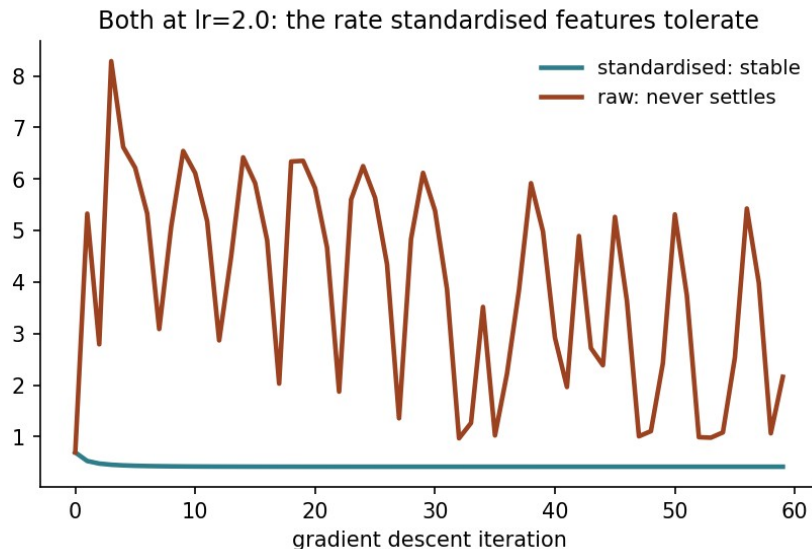
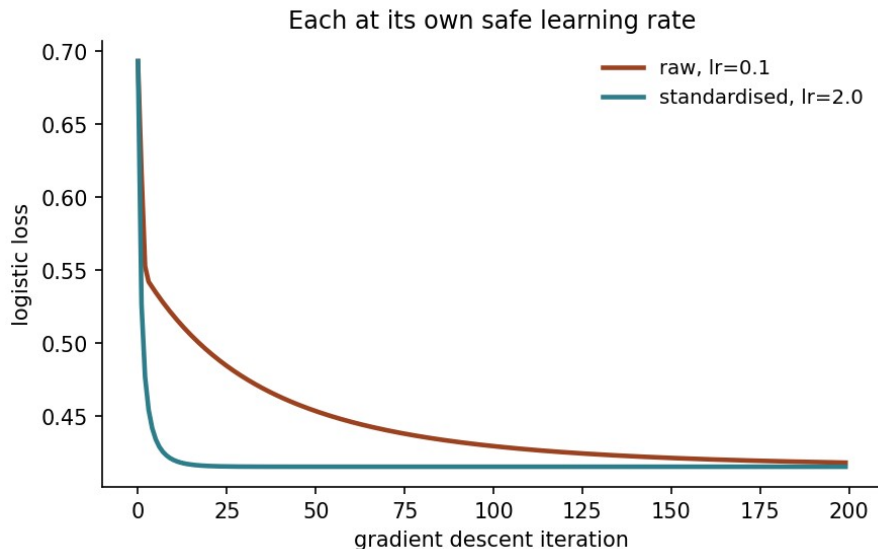
Unequal scales: a narrow valley

One stride length, two very different directions



A 100:1 variance ratio, and what it costs

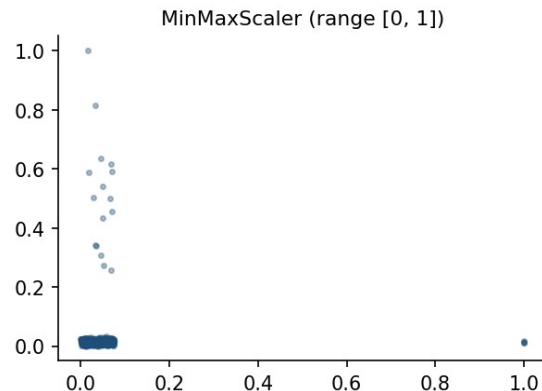
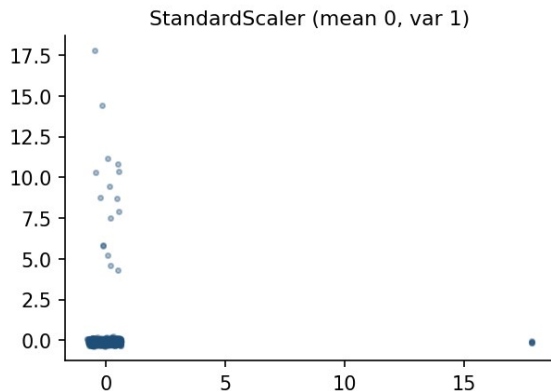
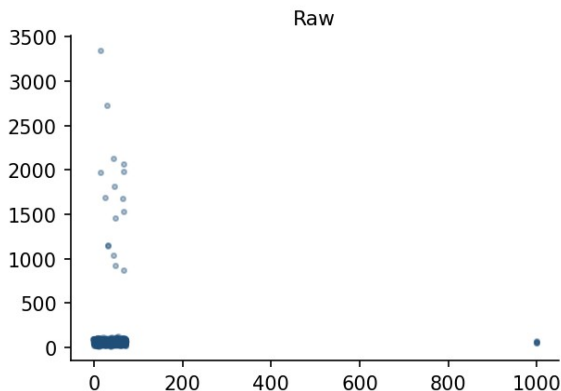
A 100:1 variance ratio, and what it costs



StandardScaler vs MinMaxScaler

$(x - \mu)/\sigma$ and $(x - \min)/(\max - \min)$, both fitted on training data only.

tenure_months (x) against monthly_charges (y)



Every encoding makes a claim about the categories

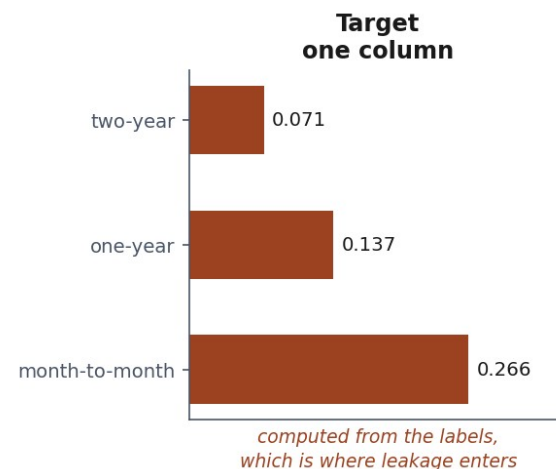
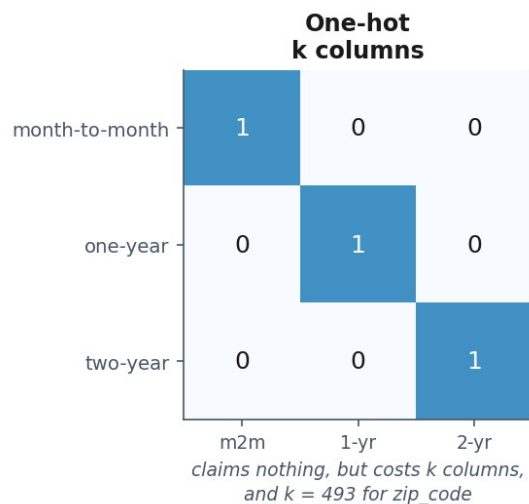
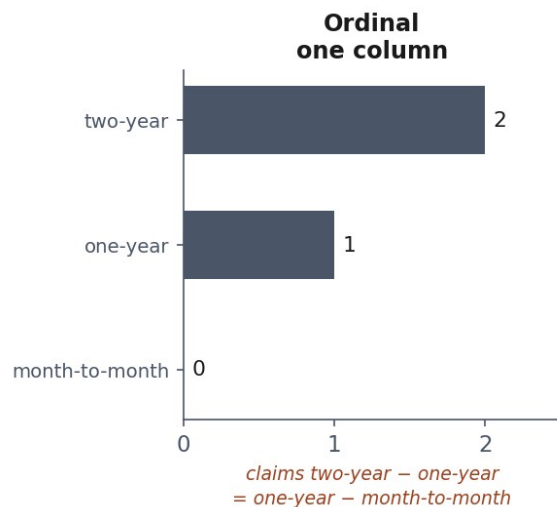
A model needs numbers, and "month-to-month" is text: encoding turns it into numbers.

contract_type	rows	churn rate
month-to-month	1,092	0.266
one-year	488	0.137
two-year	420	0.071

One customer, three encodings

Encoding	A one-year customer becomes	What that asserts
One-hot	[0, 1, 0]	all different, none closer to another
Ordinal	1	ordered, and equally spaced
Target	0.137	replaced by the churn rate of its own category

Ordinal: equal steps, unequal churn



One-hot: no order assumed, but wide

- A column with k categories becomes k columns of 0s and 1s, a single 1 per row: `OneHotEncoder`
- No order between categories, all equally far apart: that is why it is the default
- The price is width: `zip_code` would add **493** columns to 2,000 rows
- Target encoding uses one column however many categories there are, but it is computed from the target - which is how leakage gets in

One one-hot column too many

Most models include a constant column of ones, the **intercept**. The k one-hot columns (*dummies*) also always add up to 1:

$$\sum_{j=1}^k \text{dummy}_j = 1 = \text{intercept}$$

4 columns, but only rank 3

Why all k dummies plus an intercept cannot work

contract_type	m2m	1y	2y	sum	intercept
month-to-month	1	0	0	= 1	1
one-year	0	1	0	= 1	1
two-year	0	0	1	= 1	1

the k dummy columns always sum to the intercept column: rank deficient by exactly one

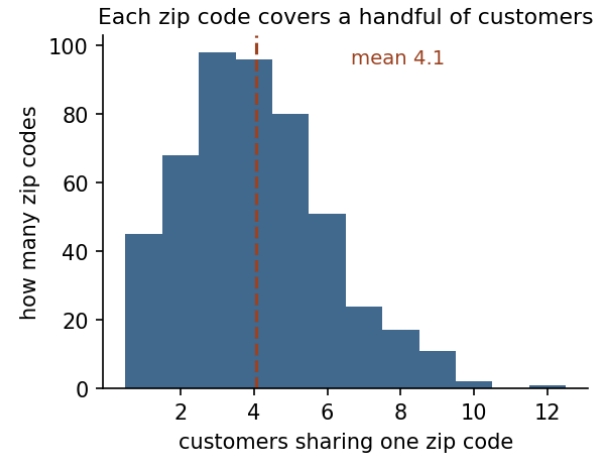
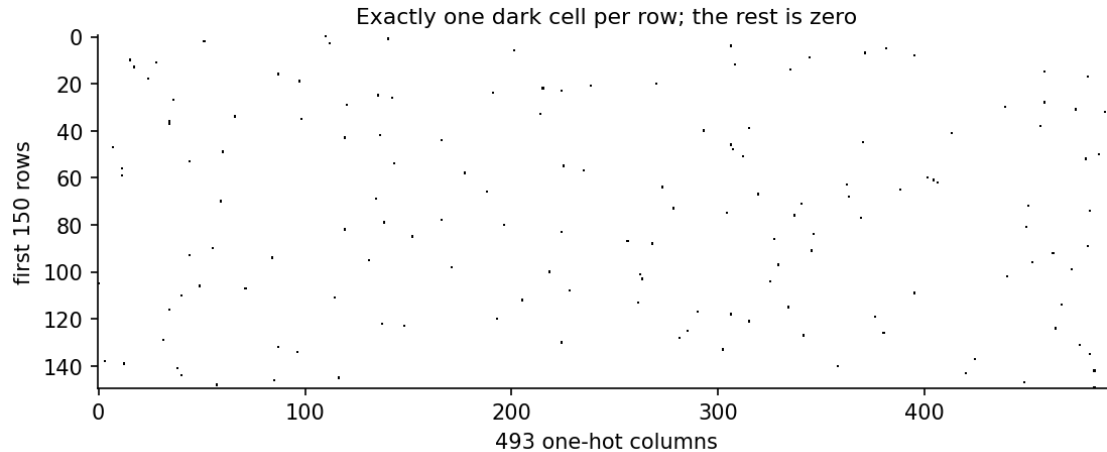
The curse of dimensionality, briefly

One-hot `zip_code`: **493** new columns, mostly zero.

- Methods based on distances (Lesson 6): all points end up roughly equally far apart
- More coefficients to estimate from the same 2,000 rows: noisier estimates

zip_code: 493 codes, 2,000 customers

zip_code, one-hot encoded: 493 columns for 2000 rows



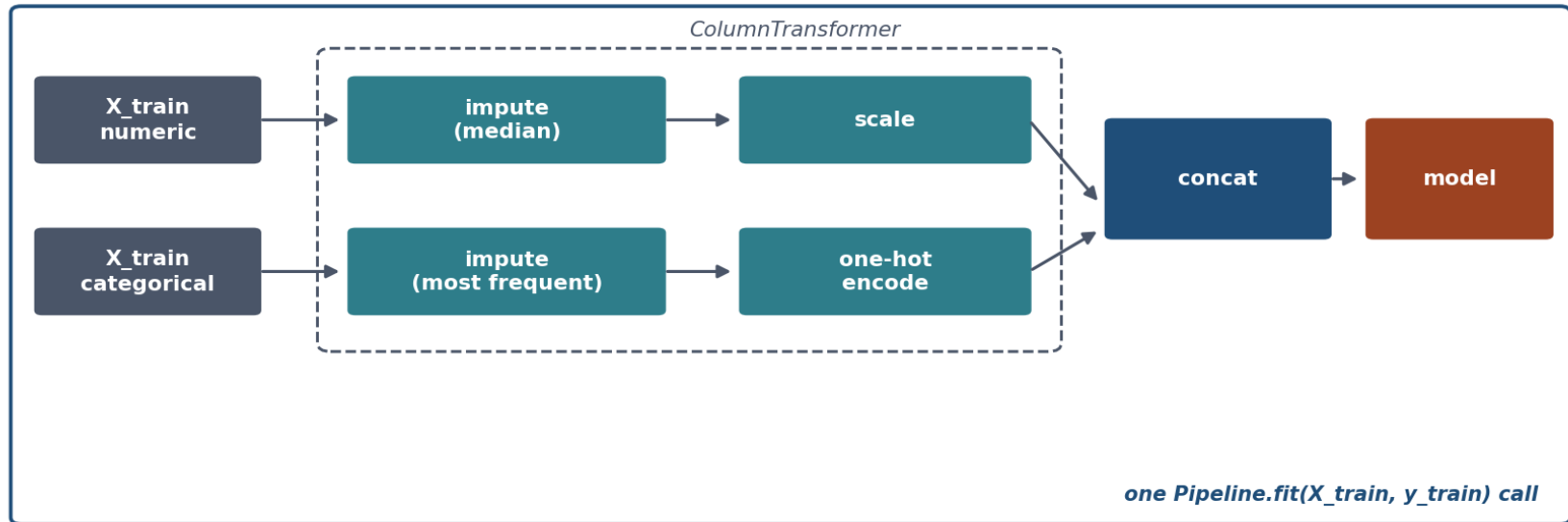
Preprocessing is part of f

f is the whole pipeline: **every** number learned from data - the scaler's means, the imputer's fill values - not only the model's coefficients.

$$\mathbb{E}_{T \sim \mathcal{D}^m} [\hat{R}_T(f)] = R(f)$$

ColumnTransformer + Pipeline

Every learned parameter comes from inside this box



The code

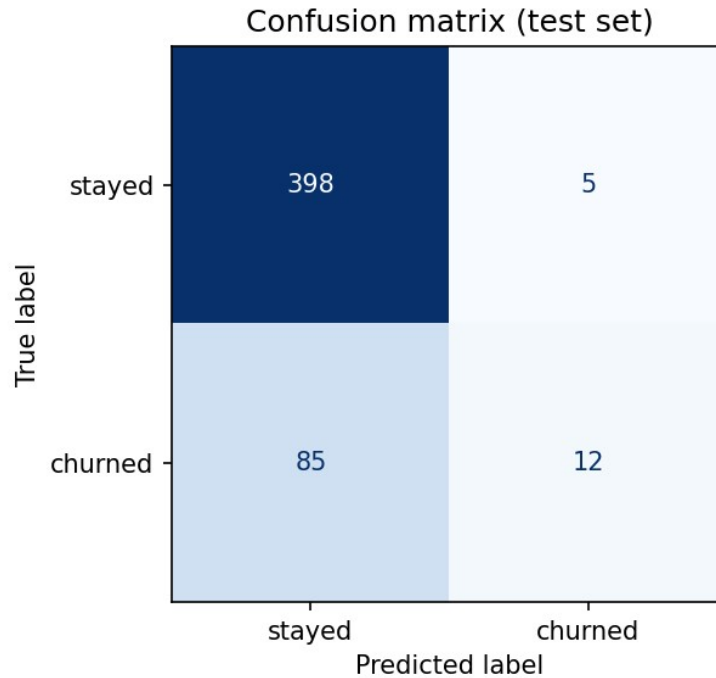
```
prep = ColumnTransformer([  
    ("num", num_pipe, num_cols),  
    ("cat", cat_pipe, cat_cols),  
])  
model = Pipeline([  
    ("prep", prep),  
    ("clf", clf),  
])  
model.fit(X_train, y_train)
```

The model, on churn

- Accuracy: baseline **0.806**, model **0.820** (numeric + low-cardinality categoricals, no zip)
- Area under the receiver operating characteristic curve (AUC): **0.751**, the chance a random churning scores above a random non-churner; 0.5 is a coin flip
- In code: `LogisticRegression` against `DummyClassifier`, `roc_auc_score`

Accuracy barely beats the baseline. Why?

Where the errors fall



Feature engineering

New columns built from existing ones, which a linear model cannot build itself.

`total_paid = charge × months stayed`: AUC
0.7514 → 0.7548. Fix the 999s in `tenure_months`
first and it scores **0.7439**: the gain came from the
errors.

In code: `FunctionTransformer(add_total_paid)`,
inside the pipeline

Notebook 2, live

section, and its slides	you call → the number to come back with
why scale, 24-27	gradient descent by hand → ratio 110:1 (built as 100:1); rate 2.0 scaled, 0.1 raw; raw at 2.0 swings 0.69 to 8.29
encoding, 28-33	pd.get_dummies, np.linalg.matrix_rank → 4 columns, rank 3
the pipeline, 36-38	train_test_split, then ColumnTransformer + Pipeline: SimpleImputer, StandardScaler, OneHotEncoder, LogisticRegression
the result, 39-40	DummyClassifier, accuracy_score, roc_auc_score, ConfusionMatrixDisplay
feature engineering, 41	FunctionTransformer → 0.7514 → 0.7548 → 0.7439

Same rule, broken three ways

Same rule, broken three ways: only the first one looks like a bug

Lesson 1

select features on the full dataset

visible: an obvious extra step

Today, leak 1

impute a missing value on the full dataset

invisible: looks like ordinary cleaning

Today, leak 2

encode a category on the full dataset

invisible: two unremarkable lines of code

Leak 1: impute before splitting

`KNNImputer` fills a gap with the average of the five most similar rows: its **donors**.

Fit it on training and test rows together, and some donors are test rows: test information enters the training data.

The smoking gun

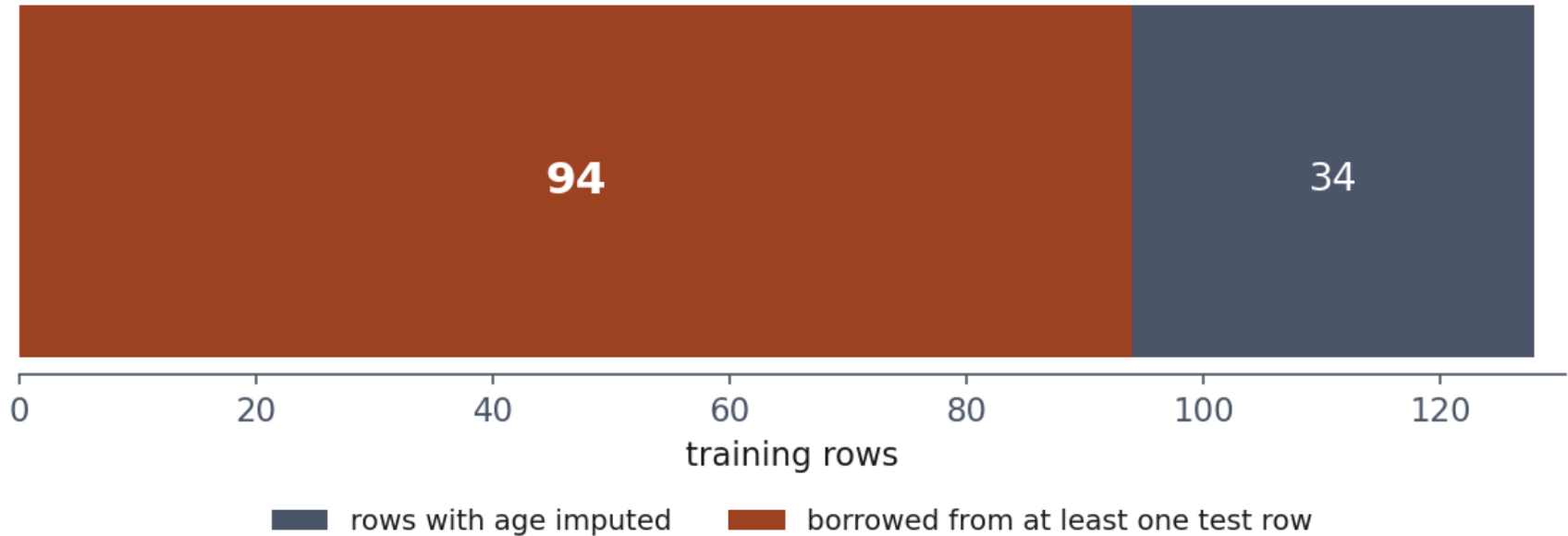
94 of 128 training rows with missing age had a **test-set row** among the five donors used to fill it.

In code: `nan_euclidean_distances`, the distance the imputer itself uses, finds the donors.

Nothing raised an error.

Leakage, counted

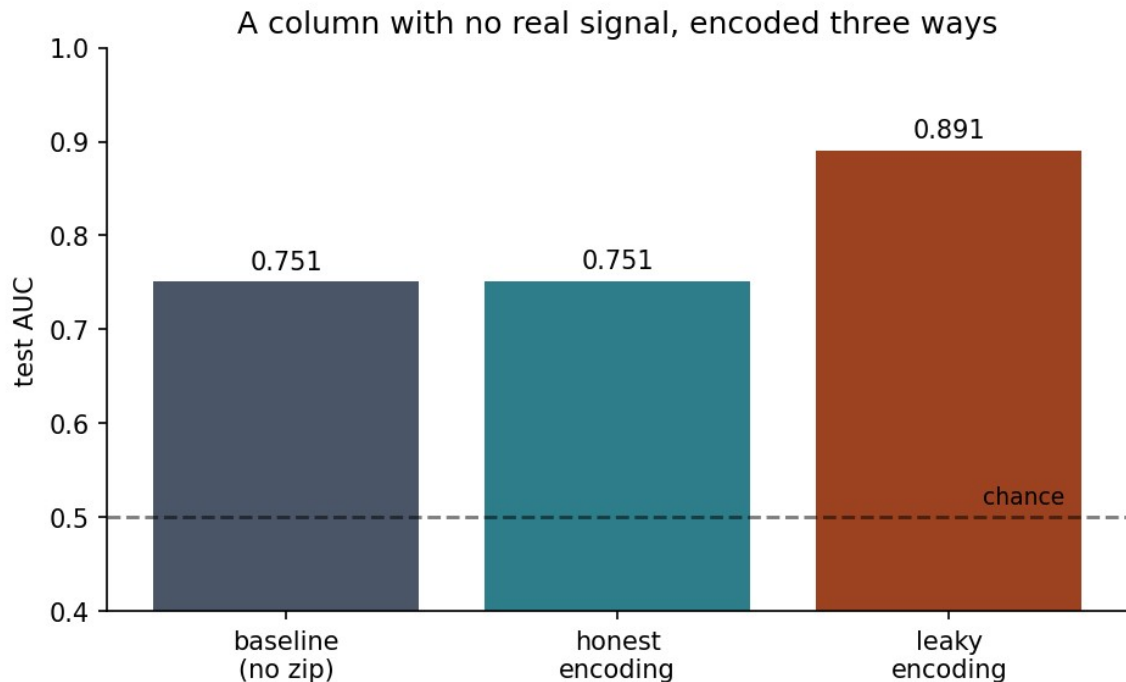
94 of 128 imputed values touched the test set



Leak 2: encode before splitting

Replace `zip_code` with the **average churn rate of its own customers**, computed on all 2,000 customers, before the split.

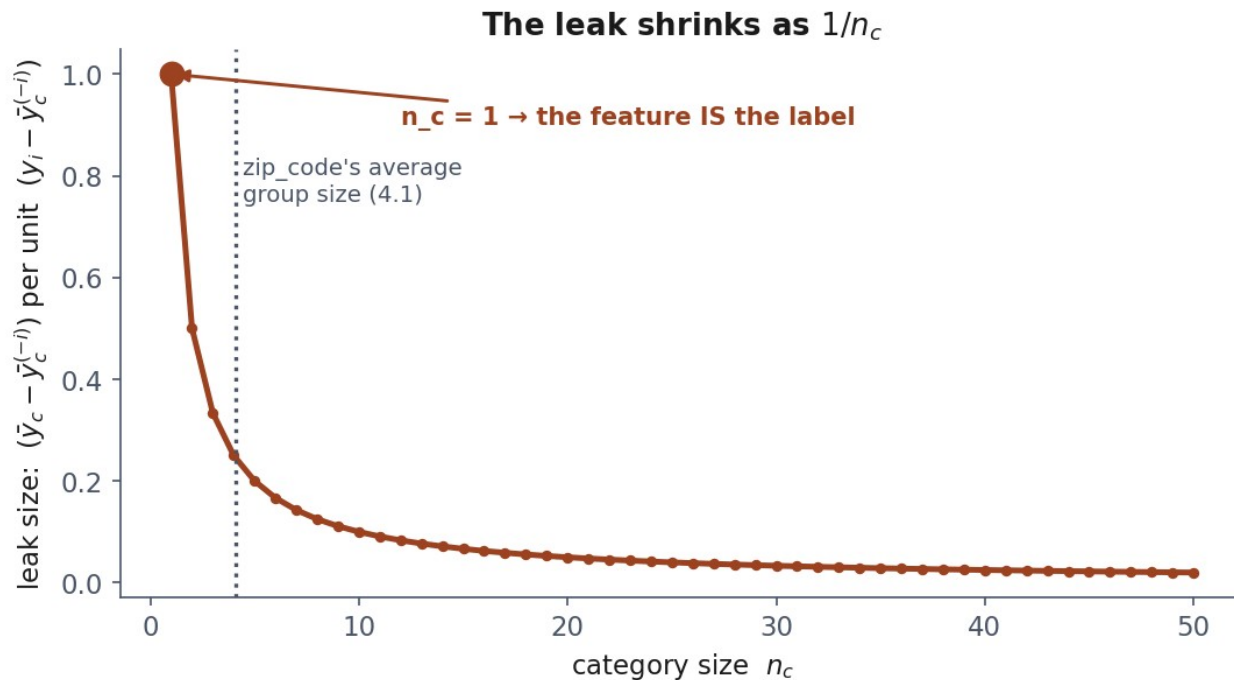
A column with no real signal, encoded three ways



How much of its own label a row sees

$$\bar{y}_c - \bar{y}_c^{(-i)} = \frac{y_i - \bar{y}_c^{(-i)}}{n_c}$$

The leak shrinks as $1/n_c$



The fix

`sklearn.preprocessing.TargetEncoder`, inside the pipeline.

It splits the training rows into five parts and encodes each row from the **other four**: never from its own label.

The rule, restated

Everything that **learns from data** is fitted on the training rows only, inside the pipeline.

For example: a mean, a median, the nearest neighbours, a churn rate per category, the edges of bins.

Notebook 3, live

leak, and its slides	you call → what it shows
imputation, 44-46	KNNImputer fitted on every row; nan_euclidean_distances, its own metric, names the donors → 94 of 128
encoding, 47-50	churn rate per zip_code, before train_test_split → AUC 0.891 from a column with no signal
the fix, 51	TargetEncoder inside the ColumnTransformer → the honest 0.751
your own split	clone both models, refit on a split of your own
scored with	roc_auc_score throughout

What we did today

- Explored before transforming: types, gaps, shape, correlations
- Diagnosed *why* values are missing, not just *how many*
- Two outlier rules, and what neither can tell you
- Scaling, encoding, engineering: all inside Pipeline
- Two leaks that raise no error and look like good results

Homework: we discuss it on Friday 16 October

Exercises/

02_data_exploration_and_preparation.md

Build the full preparation pipeline yourself, and **find a leak on purpose** before fixing it.

Before next week

- Work the three notebooks in order
- Read the handout: it has the derivations behind today's slides
- Take the quiz
- **Do the exercise**

Next: regression, the first model we derive completely.